Literature Survey — SQL Injection Detection and Protection

> Working manuscript for the DAS PE4 report. Prose follows the departmental *Instructions for writing a literature survey*: methodology in the **past tense**, each study's limitation in the **present tense**, author–year citation style, a five-part paragraph per paper, and a comparative table. All ten primary studies are peer-reviewed articles published between 2023 and 2026 in approved venues (IEEE, Elsevier/ScienceDirect, Springer, MDPI, Wiley).

---

1. Introduction and scope

Structured Query Language injection (SQLi) remains one of the most damaging classes of web-application vulnerabilities and has featured in the OWASP Top-10 "Injection" category for over a decade. An SQLi flaw arises whenever untrusted input is concatenated into a query that a database interpreter then parses as code, allowing an attacker to alter query logic to bypass authentication, exfiltrate data, or corrupt records. Because the root cause is a *confusion of code and data*, effective protection is layered rather than singular: it spans secure-coding controls (parameterized queries, allow-list input validation, object–relational mapping, least-privilege accounts, and credential hashing) and runtime controls (signature- and learning-based web-application firewalls, and network-flow monitoring). This survey reviews ten recent studies that, together, cover the two dominant research fronts — *learning-based detection* and *deployable prevention* — and it isolates the gaps that motivate a defence-in-depth prototype implementing more than one protection variant.

2. Reviewed studies (2023–2026)

Crespo-Martínez et al. (2023) addressed the problem that most network-level SQLi detectors must inspect every packet, which is impractical for routers carrying heavy traffic. The authors gathered two datasets of network-flow records captured from SQLi attacks launched against the most popular database engines, deliberately using lightweight flow protocols rather than full payload capture. They evaluated several supervised machine-learning algorithms on these flow features and selected a Logistic-Regression model as the best performer after comparative testing. Crespo-Martínez et al. reported a detection rate above 97% with a false-alarm rate below 0.07% on flow data alone. The approach is limited by its reliance on coarse flow-level features, which omit the query payload and therefore constrain the identification of obfuscated or previously unseen injection variants.

Alghawazi, Alghazzawi, and Alarifi (2023) investigated deep sequence learning for detecting SQLi payloads in application input. They trained their model on a publicly available dataset of benign and malicious SQL statements, treating each query as a token sequence. The authors designed a recurrent neural network autoencoder for representation learning and benchmarked it against seven

conventional models, including ANN, CNN, decision tree, naïve Bayes, SVM, random forest, and logistic regression. Alghawazi et al. reported an accuracy of 94% and an F1-score of 92%, outperforming the compared baselines on their dataset. The method is limited by a moderate accuracy ceiling and by its evaluation on a single public corpus, which leaves generalization to real-world traffic uncertain.

Thalji et al. (2023) targeted the automation of feature engineering for SQLi detection, arguing that hand-crafted features constrain accuracy. They used a preprocessed corpus of 46,392 SQL queries and applied an autoencoder network, "AE-Net," to learn high-level deep features directly from the query text. These learned features were then fed to several supervised classifiers, and the models were refined through hyperparameter tuning and validated with k-fold cross-validation and a statistical t-test. Thalji et al. reported that an extreme gradient-boosting classifier achieved a k-fold accuracy of 0.99, exceeding the prior studies they compared against. The approach is limited by its evaluation on a curated textual dataset without real-time or adversarial testing, and by the additional overhead its feature-extraction stage introduces.

Alotaibi and Vassilakis (2023) explored whether software-defined networking (SDN) could host a web-application firewall (WAF) capable of blocking SQLi. They implemented a proof-of-concept in which the SDN controller inspected web traffic, using two complementary detection methods — attack signatures and regular expressions — and they compared the system against the traditional ModSecurity WAF. The authors measured both detection capability and performance overhead on the controller. Alotaibi and Vassilakis demonstrated that the SDN controller successfully detected and blocked SQLi requests and achieved lower TCP-ACK latency than ModSecurity, which in turn imposed slightly lower controller overhead. The design is limited by its signature- and regex-based matching, which does not generalize to obfuscated or novel payloads, and by the centralized controller acting as a potential scalability bottleneck.

Arasteh et al. (2024) framed SQLi detection as a feature-selection problem, noting that identifying the most informative features is NP-complete. They prepared a training dataset of thirteen numeric features extracted from SQL statements and developed two binary variants of the Gray-Wolf Optimizer to select the most effective subset. The reduced feature sets were then supplied to several machine-learning classifiers for evaluation. Arasteh et al. reported 99.68% accuracy, 99.40% precision, and 98.72% sensitivity while using only about 20% of the original features, indicating gains in both accuracy and efficiency. The method is limited by the computational cost and initialization sensitivity of the metaheuristic search, and by validation on a single engineered dataset that constrains generalization.

Paul, Sharma, and Olukoya (2024) argued that the literature treats SQLi as a binary, single-vector detection task and neglects prioritization and prevention. They assembled a large corpus of 457,233 benign and malicious traffic samples, of which 70,023 contained SQLi payloads, and reformulated the task as a multi-class, multi-attack-vector problem. The authors proposed "SQLR34P3R," which combined a hybrid CNN–LSTM detector with a risk-analysis stage that incorporated CVE and vulnerability data to prioritize remediation, and they evaluated it in real-time scenarios against state-of-the-art alternatives. Paul et al. reported an average F1-score of 97% for web-traffic filtering while adding a prioritization capability absent from earlier work. The solution is limited by its

dependence on the currency of external CVE data and by the computational overhead of a hybrid deep model under real-time constraints.

Qu et al. (2024) examined the robustness of commercial "WAF-as-a-service" offerings, whose security they observed to be largely unstudied despite wide adoption. They developed "AdvSQLi," an attack framework that applied a minimal series of semantics-preserving transformations to the hierarchical parse-tree of an SQLi payload so that the mutated payload retained its malicious effect while evading detection. The authors generated adversarial payloads from two public datasets and tested them against machine-learning detectors and seven mainstream cloud WAF services under black-box conditions. Qu et al. reported a maximum attack-success rate of 100% against ML-based detectors and found every evaluated commercial WAF bypassable — for example, exceeding 79% success against the F5 WAF. As an offensive study, the work is limited in that it exposes but does not remedy the weakness, underscoring that robust defences against adversarial mutation remain lacking.

Bakır (2025) proposed a unified detector for the two most prevalent injection-style web attacks, cross-site scripting (XSS) and SQLi. The study drew on three benchmark datasets and represented each input through a fusion of natural-language-processing embeddings, combining Word2Vec, the Universal Sentence Encoder, and FastText features. This fused representation, named "UniEmbed," was supplied to several machine-learning classifiers, with a multi-layer perceptron and a hard-voting ensemble performing best. Bakır reported an F1-score of 0.9980 and accuracy above 0.9980 for SQLi, and comparably high scores for XSS, exceeding the individual embedding methods. The approach is limited by the high dimensionality and compute cost of triple-embedding fusion and by offline benchmark evaluation without adversarial or real-time testing.

Floris et al. (2025) confronted the adversarial-robustness gap directly in the context of the widely used ModSecurity WAF and its OWASP Core Rule Set (CRS), whose manually chosen rules and weights they found to give a suboptimal detection/false-alarm trade-off. The authors used machine learning to automate selection of the CRS rule subset and their weights for a monitored service, and they applied adversarial training to harden the resulting classifier against iteratively refined SQLi payloads. They evaluated the system, "ModSec-AdvLearn," on ModSecurity with the CRS rules. Floris et al. reported up to a 30% increase in detection rate with negligible false alarms while discarding up to half of the CRS rules, and an improvement of up to 85% in robustness against adversarial SQLi. The method is limited to SQLi on CRS-based WAFs, and its adversarial robustness is bounded by the coverage of the attack model used during training.

Ahmed and Al-dabag (2026) addressed the deployment gap between high offline detection scores and practical real-time prevention. They merged, cleaned, and deduplicated several public SQLi datasets to expose the model to diverse query patterns, then encoded queries with a fusion of TF-IDF, Word2Vec, and FastText features compressed through a supervised autoencoder. The authors used a genetic algorithm to optimize the hyperparameters of Random Forest, SVM, decision-tree, and MLP classifiers, and they embedded the best model inside a lightweight Flask-based WAF for real-time HTTP inspection and blocking, testing it with SQLMap v1.8.4. Ahmed and Al-dabag reported that the MLP achieved 0.9987 accuracy and that the deployed firewall provided effective real-time prevention beyond offline benchmarks. The framework is limited by its dependence on training-data coverage of evolving payloads and by validation on a single deployed application.

3. Comparative summary

#	Author(s)	Year	Venue (publisher)	Method / classifier	Key contribution	Study limitation (present tense)
1	Crespo-Martínez et al.	2023	Computers & Security (Elsevier)	Logistic Regression on network-flow features	SQLi detection from lightweight flow data (>97% DR, <0.07% FAR)	Reliance on flow-level features is limiting for obfuscated/novel payloads
2	Alghawazi, Alghazzawi & Alarifi	2023	Mathematics (MDPI)	RNN autoencoder vs. 7 baselines	Deep sequence model for SQLi text (94% acc, 92% F1)	Moderate accuracy and single-dataset scope are limiting
3	Thalji et al.	2023	IEEE Access	Autoencoder features + XGBoost	Automated deep feature engineering (0.99 k-fold acc)	Lack of real-time/adversarial testing is limiting
4	Alotaibi & Vassilakis	2023	Future Internet (MDPI)	SDN controller WAF (signatures + regex)	SDN-hosted WAF competitive with ModSecurity	Signature/regex matching is limited against unseen payloads
5	Arasteh et al.	2024	Neural Computing & Applications (Springer)	Binary Gray-Wolf Optimizer + ML	Metaheuristic feature selection (99.68% acc, ~20% features)	Search cost and single-dataset scope are limiting
6	Paul, Sharma & Olukoya	2024	J. Information Security & Applications (Elsevier)	Hybrid CNN–LSTM + CVE risk analysis	Detection + prioritization + prevention (97% F1)	Dependence on CVE currency and real-time overhead is limiting
7	Qu et al.	2024	IEEE Trans. Information Forensics & Security	Parse-tree adversarial mutation (AdvSQLi)	Bypasses 7 commercial WAFs (up to 100% ML, >79% F5)	Exposes but does not remedy the weakness; robust defence is lacking
8	Bakır	2025	Arabian J. Science & Engineering (Springer)	Word2Vec+USE+FastText fusion + MLP	Unified XSS/SQLi detector (0.998 F1)	Fusion compute cost and offline-only evaluation are limiting
9	Floris et al.	2025	IEEE Trans. Information Forensics & Security	ML rule selection + adversarial training on ModSecurity/CRS	Adversarially robust WAF (+30% DR, +85% robustness)	Scope limited to SQLi/CRS; bounded by training attack model
10	Ahmed & Al-dabag	2026	Computers (MDPI)	TF-IDF+W2V+FastText → SAE, GA-tuned MLP, Flask WAF	Deployment-oriented real-time detection+prevention (0.9987 acc)	Coverage of evolving payloads and single-app validation are limiting

4. Research gap and objective

4.1 Synthesis of gaps

Reading the ten studies together reveals a field that has largely *solved offline detection* — reported accuracies cluster at 0.94–0.99 — yet leaves four gaps open that a protection-focused project must confront:

1. **Adversarial-robustness gap.** Qu et al. (2024) demonstrated that parse-tree-level mutations bypass both ML detectors and every tested commercial WAF, and Floris et al. (2025) only partially closed this for CRS-based firewalls. Detection accuracy on clean data therefore over-states real security.
2. **Generalization / concept-drift gap.** Most detectors (studies 2, 3, 5, 8) were validated on a single curated corpus and are limited against obfuscated or previously unseen payloads and evolving SQL dialects.
3. **Detection-versus-prevention gap.** The literature skews heavily toward *detecting* payloads; only Paul et al. (2024) and Ahmed and Al-dabag (2026) integrate detection into a real-time *prevention* pipeline, and neither evaluates classical secure-coding controls (parameterization, allow-listing, least privilege) alongside the learning layer.
4. **Deployment-cost gap.** High-accuracy models (hybrid deep networks, triple-embedding fusion) carry latency and resource overheads that are limiting for in-line deployment, a tension the flow-based work of Crespo-Martínez et al. (2023) highlights from the opposite direction.

The through-line is that *no single control is sufficient*: secure-coding controls neutralize the root cause but are unevenly applied in real codebases, while learning-based WAFs generalize but remain adversarially fragile and costly. The literature rarely evaluates the two families **together** as a layered defence.

4.2 Objective

Motivated by these gaps, the objective of this project is to **design, implement, and empirically evaluate a defence-in-depth scheme for SQL-injection protection that combines more than one protection variant** — parameterized queries, allow-list input validation, ORM-based access, salted credential hashing, and a transparent signature WAF — within a single reproducible login application, and to demonstrate, on a common set of conventional and comment/UNION/tautology payloads, that (i) the vulnerable baseline is trivially bypassed, (ii) each individual control blocks the attack while preserving legitimate authentication, and (iii) the layered configuration provides the strongest coverage at a measured, sub-millisecond query-time cost. This objective converts the surveyed gaps into a concrete, testable artefact — the accompanying *SQLi Defence Lab* prototype — and frames future work on adaptive, adversarially robust detection as the next layer to be added.

References (approved venues, 2023–2026)

1. Crespo-Martínez, I. S., Campazas-Vega, A., Guerrero-Higueras, Á. M., Riego del Castillo, V., Álvarez-Aparicio, C., & Fernández-Llamas, C. (2023). SQL injection attack detection in network flow data. *Computers & Security, 127*, 103093. https://doi.org/10.1016/j.cose.2023.103093
2. Alghawazi, M., Alghazzawi, D., & Alarifi, S. (2023). Deep learning architecture for detecting SQL injection attacks based on RNN autoencoder model. *Mathematics, 11*(15), 3286. https://doi.org/10.3390/math11153286
3. Thalji, N., Raza, A., Islam, M. S., Samee, N. A., & Jamjoom, M. M. (2023). AE-Net: Novel autoencoder-based deep features for SQL injection attack detection. *IEEE Access, 11*, 135507–135516. https://doi.org/10.1109/ACCESS.2023.3337645
4. Alotaibi, F. M., & Vassilakis, V. G. (2023). Toward an SDN-based web application firewall: Defending against SQL injection attacks. *Future Internet, 15*(5), 170. https://doi.org/10.3390/fi15050170
5. Arasteh, B., Aghaei, B., Farzad, B., Arasteh, K., Kiani, F., & Torkamanian-Afshar, M. (2024). Detecting SQL injection attacks by binary gray wolf optimizer and machine learning algorithms. *Neural Computing and Applications, 36*, 6771–6792. https://doi.org/10.1007/s00521-024-09429-z
6. Paul, A., Sharma, V., & Olukoya, O. (2024). SQL injection attack: Detection, prioritization & prevention. *Journal of Information Security and Applications, 85*, 103871. https://doi.org/10.1016/j.jisa.2024.103871
7. Qu, Z., Ling, X., Wang, T., Chen, X., Ji, S., & Wu, C. (2024). AdvSQLi: Generating adversarial SQL injections against real-world WAF-as-a-service. *IEEE Transactions on Information Forensics and Security, 19*, 2623–2638. https://doi.org/10.1109/TIFS.2024.3350911
8. Bakır, R. (2025). UniEmbed: A novel approach to detect XSS and SQL injection attacks leveraging multiple feature fusion with machine learning techniques. *Arabian Journal for Science and Engineering, 50*(19), 15591–15604. https://doi.org/10.1007/s13369-024-09916-4
9. Floris, G., Scano, C., Montaruli, B., Demetrio, L., Valenza, A., Compagna, L., Ariu, D., Piras, L., Balzarotti, D., & Biggio, B. (2025). ModSec-AdvLearn: Countering adversarial SQL injections with robust machine learning. *IEEE Transactions on Information Forensics and Security, 20*. https://doi.org/10.1109/TIFS.2025.3583234
10. Ahmed, S. S., & Al-dabag, M. L. (2026). Deployment-oriented multi-embedding machine-learning framework for SQL injection detection and prevention in a web application firewall. *Computers, 15*(6), 368. https://doi.org/10.3390/computers15060368